



UNIVERSITI MALAYSIA TERENGGANU

FACULTY OF COMPUTER SCIENCE AND MATHEMATICS

CSE3433
Software Architecture

Cloud-Native Learning Management System (LMS)
(Cloud-Native / Layered Architecture)

Prepared by:

Group No.5

No.	Name	Matric No.
1	NUR MAS QASRINA BINTI MAS JAYA	S74706
2	AIDA HUSNA BINTI MUHAMAD ZAIDI	S76103
3	NURSYAHEERA BINTI MOHD HISHAM	S76276

Prepared for:
Dr Mohd Nor Hassan

COMPUTER SCIENCE (SOFTWARE ENGINEERING) WITH HONOUR
SEMESTER II 2025/2026

Table Of Content

1. PROBLEM DESCRIPTION.....	3
2. SYSTEM OBJECTIVES.....	3
3. FUNCTIONAL REQUIREMENT.....	4
4. SELECTED ARCHITECTURE STYLE.....	5
5. TEAM MEMBERS ROLES AND RESPONSIBILITIES.....	7
6. REFERENCES.....	7

1. PROBLEM DESCRIPTION

Current Cloud-Native LMS platforms still face several issues that affect both system performance and user experience. One of the main problems is the monolithic architecture, where all components are tightly connected, making it difficult to update or modify one part without affecting the whole system. These systems also have limited scalability, which leads to slow performance during peak usage such as exams or assignment submissions, and they may experience downtime due to limited infrastructure. In addition, poor system integration and weak API support make it hard to connect with external tools, reducing overall flexibility.

These issues impact all main users of the system, which are students, lecturers, and administrators. Students may struggle to access learning materials, miss deadlines due to system issues, and have a poor learning experience without real-time updates. Lecturers may face difficulties in managing courses, grading, and tracking student performance efficiently, which increases their workload. Administrators also face challenges in maintaining and scaling the system, higher operational costs, and difficulties in adding new features or integrating external systems.

2. SYSTEM OBJECTIVES

- a. To develop a scalable and modular cloud-native Learning Management System (LMS).

The LMS shall be designed in a cloud native manner with a system architecture that allows modular component development using APIs.

- b. To ensure the efficiency of the interaction between system components through API-based communication.

The interaction between the front-end, the backend service, and other external modules of the system shall be possible through communication using APIs.

- c. To provide the availability and reliability of the learning platform.

The system shall be designed to minimize downtime and ensure continuous access to learning resources through distributed deployment and cloud

infrastructure capabilities.

- d. To provide cross-platform capability for the learning platform.

The system shall allow users (students and lecturers) to access the platform via web browsers across multiple devices without requiring platform-specific installation.

- e. To enable easy maintainability and scalability of the system architecture.

The system shall be structured into loosely coupled modules to allow easier maintenance, debugging, and future enhancement without affecting the entire system.

3. FUNCTIONAL REQUIREMENT

- a. User Management (Login/Register)

- i. User Registration: The system shall allow users to register with unique credentials and securely authenticate using industry-standard protocols.

- ii. Role-Based Access: The system shall assign student, instructor, or admin roles, ensuring users only access features relevant to their permissions.

- b. Course Management

- i. Course Lifecycle: Instructor shall have the ability to create, update, and archive courses, including managing their descriptions and associated metadata.

- ii. Curriculum Organization: The system shall enable instructors to upload and structure content formats such as PDF, video, and text into organized modules.

- c. Assignment & Submission

- i. Assignment Management: Instructors shall have the ability to define assignment parameters such as instructions, points, and deadlines.

- ii. Digital Submission: The system shall provide a secure interface for students to upload assignment files. Each submission must be timestamped to verify against the deadline.

d. Notification System

- i. Event Notification: The system shall automatically notify users of key activities, including new materials or posted grades.
- ii. Deadline Reminders: Students shall receive automated reminders 24-48 hours before assignment deadlines.

e. Progress Tracking

- i. Completion Monitoring: The system shall track and record the status of individual lessons, assignments, and update students' overall completion percentage in real time.
- ii. Visual Indicators: Students shall see progress bars and status icons on their dashboard to reflect completed and pending tasks.

4. SELECTED ARCHITECTURE STYLE

The selected architecture style for this project is Cloud-Native Architecture combined with Layered Architecture supported by microservices concepts. This architecture is chosen because it best suits the requirements of a modern Learning Management System (LMS), where many users need continuous access to learning resources, assignment submissions, and real-time progress tracking.

a. Cloud-Native Architecture

This architecture provides high scalability and reliability. The traditional monolithic LMS platforms frequently experience performance problems during times of high usage such test weeks or assignment deadlines. The suggested solution may maintain system availability with little downtime and expand resources dynamically based on user demand by utilizing cloud-native technologies. (robvet, n.d.)

b. Layered Architecture

This architecture is selected to divide the system into distinct and controllable levels including the Data Layer, Application Layer, and Presentation Layer. This separation of responsibilities improves maintainability, simplifies debugging, and allows future enhancements without affecting the entire system.

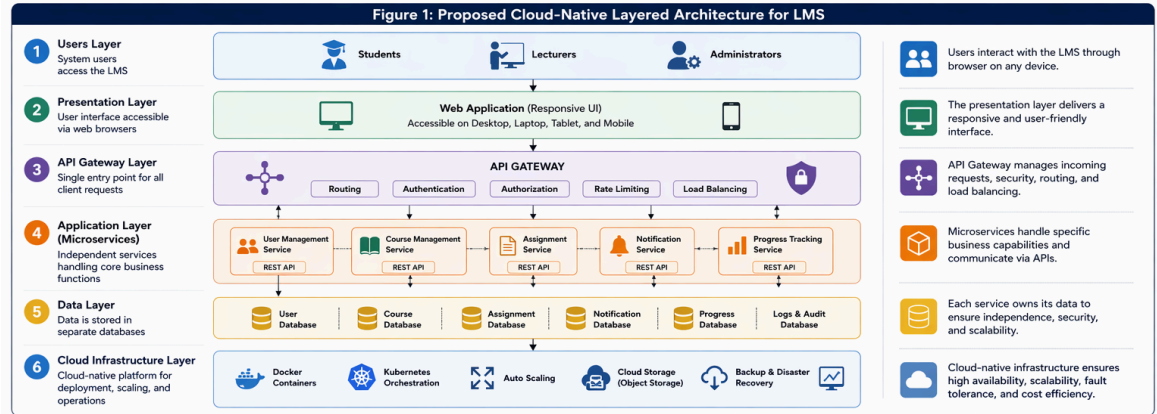


Figure 1: Proposed Cloud-Native Architecture and Layered Architecture for LMS.

c. Microservices Architecture

Microservices architecture are applied within the application layer to divide the LMS into independent modules such as User Management, Course Management, Assignment Handling, Notification, and Progress Tracking services. Each service can be developed, deployed, and updated separately, improving flexibility and reducing system complexity.

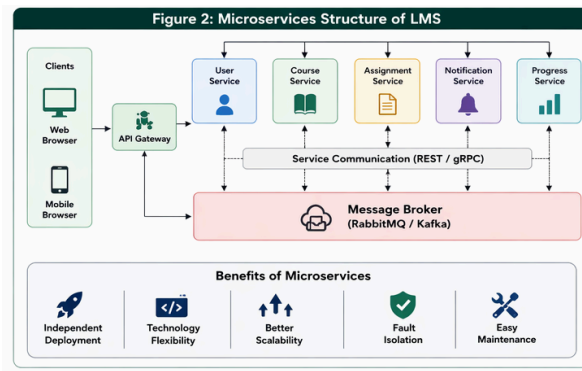


Figure 2: Microservices Structure of LMS

This selected architecture style also supports API-based communication between modules that makes it easier to integrate external tools or additional services in the future. As a result, the proposed architecture is more efficient, scalable, and future-ready compared to traditional monolithic LMS systems.

5. TEAM MEMBERS ROLES AND RESPONSIBILITIES

a. Team Structure and Roles Distribution

No.	Name	Role	Responsibilities
1.	Nur Mas Qasrina Binti Mas Jaya	Project Manager	Plan and manage overall project timeline and deliverables Coordinate team communication and task distribution Prepare architecture diagrams and documentation
2.	Aida Husna Binti Muhamad Zaidi	Backend Developer	Develop server-side logic and RESTful APIs using Node.js / Spring Boot Manage database design and integration (MongoDB / SQLite) Handle version control using GitHub
3.	Nursyaheera Binti Mohd Hisham	Fronted Developer	Develop a responsive user interface using HTML, Bootstrap, or React Integrate frontend with backend APIs Ensure usability and user experience (UI/UX) quality

6. REFERENCES

Pointer HR. (n.d.). *Software project team roles and responsibilities*. Retrieved April 11, 2026, from <https://pointer.hr/software-project-team-roles-and-responsibilities/>

LMSpedia. (2026, February 21). *LMS architecture explained: Cloud, microservices & integration guide for enterprises*. <https://lmspedia.org/lms-architecture-explained/>

Colman, H., & Colman, H. (2026, March 20). LMS Requirements Checklist: Essential features & Capabilities (2026). *iSpring knowledge hub: practical insights to evaluate LMS solutions – Boost your L&D expertise. Get instructional design best practices, eLearning how to and case studies. Gain the best of your LMS and authoring tools.* <https://www.ispring.com/knowledge-hub/lms-requirement>

Microsoft. (n.d.). *What is cloud-native?* Microsoft Learn. Retrieved April 18, 2026, from <https://learn.microsoft.com/en-us/dotnet/architecture/cloud-native/definition>